Student Meal Planner

Technical Decisions

An Implementation Document for the Grubs4Scrubs Project

Date: April 2026

Author: Yao Cheng Zhou

LO mapped: LO3 Implementation (Software Quality)

What this document is

Every project involves choices. Some of them are obvious in hindsight, some aren't. This doc records the technical decisions I made while building Grubs4Scrubs, what the alternatives were, and why I went the direction I did. It's partly for portfolio evidence and partly so I can look back later and remember why things are the way they are.

Decision: React + Vite for the frontend

Chose: React 19 with Vite as the bundler. **Alternatives considered:** Vue.js, plain HTML/CSS/JS, Next.js.

I picked React because it's what Fontys teaches and what I'd already started learning through Codecademy and class projects. Vue was an option and I've heard it's simpler to get started with, but switching frameworks mid-semester didn't make sense when I was still getting comfortable with component-based thinking.

Vite over Create React App was an easy call. CRA is basically abandoned at this point (last meaningful update was 2022), and Vite's dev server is noticeably faster. Hot module replacement works instantly instead of the 2-3 second delay CRA had. I didn't seriously consider Next.js because Grubs4Scrubs doesn't need server-side rendering, and adding Next.js would mean learning its routing conventions and API routes on top of everything else.

Decision: Custom CSS over Tailwind

Chose: Custom CSS with a `ComponentName-element-subelement` naming convention. **Alternatives considered:** Tailwind CSS, CSS Modules, styled-components.

This one I learned the hard way. The project started with Tailwind. The Dashboard page had className strings like `bg-white/[0.04] border border-white/[0.06] p-6 rounded-card` on every single div. Reading the JSX meant decoding twelve utility classes just to understand what a container looked like. Debugging was painful because the styles were scattered across the markup instead of being in one place.

I ripped Tailwind out entirely around sprint 2. Every page got its own `.css` file. Each class name describes what the element is, not what it looks like. `.Dashboard-stat-card` is immediately readable. The migration took time, but after it was done, adding new pages was faster because I could see the styles at a glance.

CSS Modules would've been fine too, but the scoping wasn't worth the extra import boilerplate for a project this size. Styled-components felt like too much abstraction. I wanted plain CSS files I could open and scan.

Decision: ASP.NET Core Web API for the backend

Chose: ASP.NET Core 8 with C#. **Alternatives considered:** Node.js/Express, Python/Flask.

This wasn't really my choice. Fontys Semester 2 requires C# and .NET. But even if I'd had the option, ASP.NET Core is a solid pick for a REST API. The built-in dependency injection, the controller routing, and the middleware pipeline are well-documented. The main downside is the learning curve. I came into the semester knowing JavaScript, not C#. The first few weeks were rough.

Decision: Raw ADO.NET instead of Entity Framework

Chose: ADO.NET with SqlConnection, SqlCommand, SqlDataReader. **Alternatives considered:** Entity Framework Core (not allowed), Dapper.

The Fontys project requirement explicitly says no EF Core. So I had to use ADO.NET. Dapper was technically an option (it's just a thin wrapper around ADO.NET), but my coach wanted me to understand what's happening at the SQL level, not abstract it away.

The trade-off is real. ADO.NET means writing every SQL query by hand, mapping every column to a property manually, and repeating a lot of boilerplate. My RecipeRepository has about 150 lines of code that would be maybe 30 with EF Core. But I actually understand every line. I know exactly which queries run, when connections open and close, and what happens when a column is null. That understanding wouldn't be there if I'd used an ORM.

I wrote helper patterns as the project grew. Using `reader.GetOrdinal("ColumnName")` instead of hardcoded column indexes, for example. That came from a debugging session where I added a column and suddenly every query after it returned wrong data because the indexes shifted.

Decision: BCrypt for password hashing

Chose: BCrypt.Net-Next NuGet package. **Alternatives considered:** SHA256 + salt, Argon2, PBKDF2.

BCrypt is the standard recommendation for password hashing in 2025. It's deliberately slow (which makes brute-force attacks expensive), it generates its own salt automatically, and the .NET package makes it two lines of code: `BCrypt.Net.BCrypt.HashPassword()` and `BCrypt.Net.BCrypt.Verify()`. SHA256 alone isn't enough because it's fast (bad for passwords) and requires manual salt management. Argon2 is technically better than BCrypt but the tooling in .NET isn't as mature. PBKDF2 is built into .NET but requires more configuration and is easier to misconfigure.

Decision: JWT for authentication tokens

Chose: JSON Web Tokens with HMAC-SHA256 signing. **Alternatives considered:** Session cookies, API keys.

JWTs work well for a React frontend talking to a .NET API because the token lives on the client side. The frontend will store it (probably in memory or localStorage) and send it with every request in the Authorization header. The server doesn't need to keep session state, which simplifies the backend.

The token carries three claims: user ID, email, and username. That means the server can identify who's making a request without hitting the database on every call. The token expires after 24 hours, which is a reasonable balance between convenience (not logging in every hour) and security (not staying logged in forever).

Session cookies would've worked too, but they add complexity with CORS and same-origin policies when the frontend and backend run on different ports during development.

Decision: JSON columns for ingredients and instructions

Chose: Store ingredients and instructions as JSON strings in NVARCHAR(MAX) columns. **Alternatives considered:** Normalized tables (Ingredients table with RecipeId FK, Instructions table with RecipeId FK).

This one is a pragmatic trade-off. Ingredients are always read and written as a complete list alongside their recipe. There's no feature in the app that queries "all recipes containing pasta." If that feature existed, a normalized table with proper indexing would be the right call. But for now, JSON-in-a-column means fewer tables, fewer JOINs, and simpler repository code.

On the frontend, the JSON gets parsed with `JSON.parse()` when the recipe is loaded and converted back with `JSON.stringify()` when it's saved. The edit modal handles the conversion between the JSON array and a human-readable textarea (one ingredient per line).

Decision: Four-project solution structure

Chose: Separate class library projects for Domain, DataAccess, Business, and API. **Alternatives considered:** Single project with folders.

Separate projects mean the compiler enforces the layer boundaries. If the API project doesn't reference DataAccess, I can't accidentally use a repository in a controller. Folders inside one project rely on discipline, which I don't always have at 11 PM when a feature needs to ship.

The extra overhead is minimal. Each project has a `.csproj` file and the solution file ties them together. The dependency injection registration in Program.cs is the same either way.

Decision: Lucide React for icons

Chose: lucide-react icon library. **Alternatives considered:** Font Awesome, React Icons, custom SVGs.

Lucide icons are clean, consistent, and tree-shakeable (only the icons I import get bundled). Font Awesome is heavier and the free tier has a limited selection. React Icons bundles multiple icon sets which adds bundle size. Custom SVGs would be the lightest option but I don't want to draw icons.

What I'd decide differently next time

A few things I'd reconsider if I started over:

- **Start with the backend first.** I built frontend pages before the API existed, which meant hardcoding sample data and then reworking everything when the real data had a different shape. Building API-first would've saved rework.
- **Add SQL constraints from day one.** No UNIQUE on Email, no FOREIGN KEY constraints. They're not strictly needed for the app to work, but they'd catch bugs earlier and they're better practice.
- **Consider Dapper.** Even with the ADO.NET requirement, Dapper is close enough to raw SQL that it might've been allowed. It would've cut the repository boilerplate significantly while keeping the SQL visible.

[NOTE: If there are other decisions you made that I didn't cover, especially around deployment, testing tools, or CI/CD, add them here.]